ESC112 Tutorial 3

1 Introduction

Topics to be covered:

- string pointers and its functions
- a basic storage allocator
- sorting input lines

2 string pointers and its functions

2.1 Example 1

```
1  int main(){
2      char amessage[] = "hello";
3      char *pmessage = "hello"; //read-only memory
4      printf("%s␣%s\n", amessage, pmessage);
5
6      amessage[0] = 'C';
7      printf("%s␣%s\n", amessage, pmessage);
8
9      //amessage behaves like a const
10     //amessage = pmessage //NOT ALLOWED, COMPILATION ERROR
11
12     //pmessage[0] = 'C'; //NOT ALLOWED, RUNTIME ERROR
13
14     pmessage = amessage;
15     pmessage[0] = 'C'; //now fine
16     printf("%s␣%s\n", amessage, pmessage);
17 }
```

so in a function, if your argument is (char*line)
then you can say line[i] = c;
but then while calling the function, you have to make sure u call it on a mutable string, ie
read_line("hello") wont work!
but buf[] = "hello";
*msg = "hello"
read_line(buf) will work!!
read_line(msg) won't work!!

Remarks:

- amessage behaves like a `const char *` that holds and address to the start of a char array of length 6 (including the ending NULL char). can't be reassigned to pmessage letter. compilation error.
- however, individual chars such as `amessage[0]` or `amessage[5]` can be reassigned.
- pmessage on the other hand is assigned the constant literal "hello". that resides in read-only memory.

amessage, on the other hand is a writable array on the stack

- therefore, individual char reassignment such as pmessage[0] = 'C'; can't write, gives runtime error.
- however, pmessage is like an ussual variable that contains an address. it can be later reassigned to amessage.
- after `pmessage = amessage;`, pmessage now points to a writable allocated memory, same as amessage. So, pmessage[0] = 'C'; is valid

FIRST, THE POINTER WAS POINTING TO A STRING LITERAL. THEN IT POINTED TO A MUTABLE ARRAY, SO REASSIGNMENT WORKED!!!!

2.2 Example 2

strcpy(dest, src) takes two char * dest and src as argument and copies src to dest. it has return type char * that returns the start of dest.

```
1 char dest[100], src [] = "hello";
2 char *ptr = src + 1;
3 printf("%s␣%s\n", src, ptr);
4 printf("%s\n", strcpy(dest, src));
5 printf("%s\n", dest);
```

prints:

```
1 hello ello
2 hello
3 hello
```

Note: it is the programmer's responsibility to ensure dest has got enough bytes allocated, otherwise, runtime error.

how to implement this?

```
1 char * str_cpy(char *t, char *s){
2     char *dest = t;
3     while((*t = *s)){
4         t++;
5         s++;
6     }
7         ;
8     return dest;
9 }
```

Equivalent idiomatic form:

```
1 char * strcpy(char *t, char *s){
2     char *dest = t;
3     while(*t++ = *s++)
4         ;
5     return dest;
6 }
```

Remarks:

- `while(*t++ = *s++)` is parsed as `while(*(t++) = *(s++))` and s++ is done first, as unary ops are R associative.
- however, s++ is the post increment op. its value is the old value before increment. indirection op * is applied to it to get the char at that address.
- similarly, t++ is the post increment op. so, its value t is first applied the * operator. *t gets the value of *s.
- t and s are both increment to point to the next char.
- assignment itself has the value that was just copied. so, while loop runs till the last NULL char (having ascii value 0) will be copied.

2.3 Example 3

strcpy(dest, src) takes two char * dest and src as argument and reports their lexicographic order. 0 if two strings are equal. positive if dest is lexicographically larger and negative if smaller.

basically, scan from left until the first point of difference. report the difference of ascii values.

```
strcmp("cat", "cat")     0        (equal)
strcmp("cat", "cattle")   <0        ('\0' < 't')
strcmp("dog", "cat")     >0       ('d' > 'c')
```

```
int str_cmp(const char *s1, const char *s2){
    while(*s1 && *s2 && *s1 == *s2){
        s1++;
        s2++;
    }       ;
    return *s1 - *s2;
}
```

const is added often in such function arguments to prohibit accidental tampering.

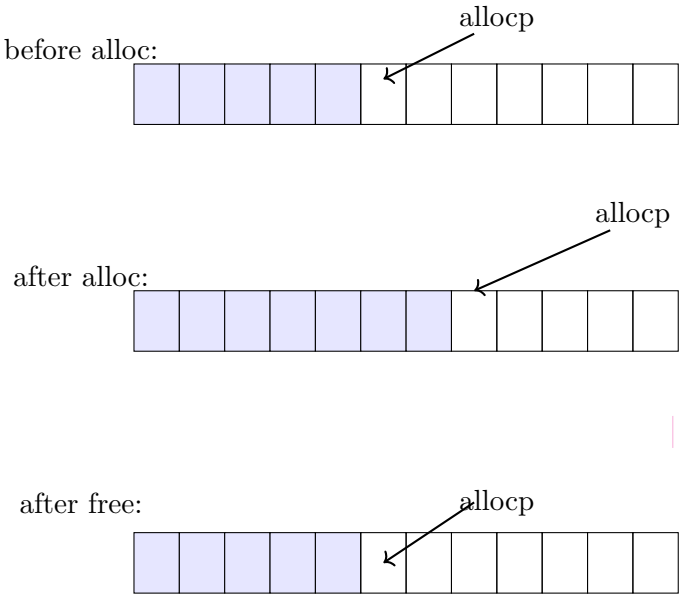
3 a basic storage allocator

we will now implement a basic version of storage allocator. this has two functionalities:

- alloc: code can request for n consecutive bytes. returns char *. takes int n.
- free: code can free its requested byte. takes char *. void return.

However, we will have a constraint for our storage allocator. all free calls must be made in the reverse of alloc calls. stdlib has a more sofisticated malloc/calloc and free that don't have this constraint.

We will keep a big pool of memory `allocbuf` and give the requesting process n bytes if it does not overflow our pool. a char * variable `allocp` will keep tract of the currently first available byte.



Now, we understand why "all free calls must be made in the reverse of alloc calls." otherwise it breaks the logic of allocp. the implementaiton follows.

```
1  #define ALLOCSIZE 10000
2
3  char allocbuf[ALLOCSIZE];
4  char *allocp = allocbuf; //or &allocbu[0].
5
6  char *alloc(int n){
7      if (allocbuf + ALLOCSIZE - 1 >= allocp + n - 1) {
8          allocp += n;
9          return allocp - n;
10     } else
11         return 0;
12 }
13
14 void afree(char *p){
15     if (p >= allocbuf && p <= allocbuf + ALLOCSIZE - 1)
16         allocp = p;
17 }
```

Remark:

- pointer inequality comparisons <, <=, >, >= are valid only within the same array. undefined across two arrays – no guarantee of behavior.
- useful for checking allocp does not overflow in alloc and p is valid in afree.

4 sorting lines of text

now, we can use our simple allocator to store a sequence of lines from input and sort them in lexicographic order.

Input:

```
Hi there
how are YOU?
I am good.
You?
Fine too.
See you.
Byeee,
```

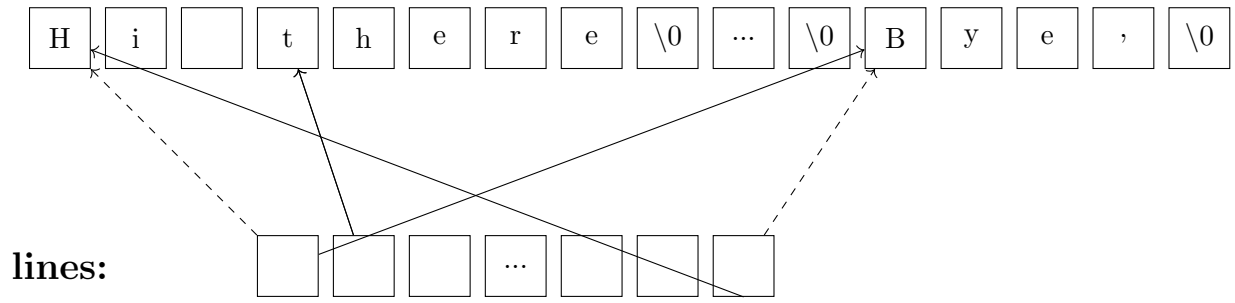
Output:

```
Byeee,
Fine too.
Hi there
I am good.
See you.
You?
how are YOU?
```

we'll use a sorting algorithm – quick sort. same algorithm will work, except we are now sorting lines of text, instead of integers, with lexicographic i.e. strcmp ordering. lines can be of variable length and unlike integers, they can't be compared or moved in a single operations. a data representation for variable length text lines is needed.

alls lines will be stored one after another in a large enough char array. allocated using alloc and then copied. an array of *char pointers* will store a char pointer for the start address of each line. during swap, no need to swap the contents. just swap the start pointers. comparison can be done by calling strcmp on the start pointers. avoids moving or copying entire strings each time.

Example: swap lines[0] and lines [6]. only pointers swap. strings don't move.



After sorting, print as per order in lines. The complete code is given below.

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h> //for exit

#define MAXLINES 100
#define LINESIZE 100
#define ALLOCSIZE 10000

char allocbuf[ALLOCSIZE];
char *allocp = allocbuf;

char * str_cpy(char *t, char *s){
    char *dest = t;
    while((*t = *s)){
        t++;
        s++;
    }
    return dest;
}

char * alloc(int n){
    if(allocp + n - 1 <= allocbuf + ALLOCSIZE - 1){
        allocp += n;
        return allocp - n;
    }
    else
        return NULL;
}

void afree(char *p){
    if(p <= allocbuf + ALLOCSIZE - 1 && p >= allocbuf)
        allocp = p;
}

int get_line(char line[], int max){
    int c, i = 0;
    while(i < max && (c = getchar()) != EOF && c != '\n')
        line[i++] = c;
    line[i] = '\0';
    return i;
}

int readlines(char *lines[], int max){
    char line[LINESIZE];
    int len, i = 0;
    while(i < max && (len = get_line(line, LINESIZE)) > 0){
        lines[i] = alloc((len + 1) * sizeof(char));
        if(lines[i] == NULL){
            printf("ERROR: not enough memory to store lines\n");
            exit(1);
        }
        str_cpy(lines[i], line);
        i++;
    }
    return i;
}
```

```c
void swap(char *lines[], int i, int j){
    char *temp = lines[i];
    lines[i] = lines[j];
    lines[j] = temp;
}

int partition(char *lines[], int n){
    char *pivot = lines[n - 1];
    int j; //head of region 3
    int i = -1; //tail of region 1, <= pivot
    for(j = 0; j < n - 1; j++){
        if(strcmp(lines[j], pivot) <= 0){
            //swap with head of region 2
            swap(lines, i + 1, j);
            i++;
        }
    }
    //swap pivot with head of region 2
    swap(lines, i + 1, j);

    //return pivot index
    return i + 1;
}

void qsortlines(char *lines[], int n){
    if(n <= 1)
        return;
    int p = partition(lines, n);
    qsortlines(lines, p);  //indices 0 to p - 1
    qsortlines(lines + p + 1, n - p - 1); //indices p + 1 to n - 1
}

void printlines(char *lines[], int num){
    for(int i = 0; i < num; i++)
        printf("%s\n", lines[i]);
}

void copy_lineptr(char *dest_lines[], char *src_lines[], int n){
    for(int i = 0; i < n; i++)
        dest_lines[i] = src_lines[i];
}

int main(){
    char *lines[MAXLINES], *init_lines[MAXLINES];
    int n = readlines(lines, LINESIZE);
    copy_lineptr(init_lines,lines, n);
    qsortlines(lines, n);
    printlines(lines, n);

    //free now
    for(int i = n - 1; i >= 0; i--){
        afree(init_lines[i]);
        init_lines[i] = NULL;    #good practice to set to NULL
    }
}
```

Alternative printlines using pointer arithmetic. since lines is itself the name of an array, it can be treated as a pointer to pointer.

```
void printlines(char *lines[], int num){
    while (num--)
        printf("%s\n", *lines++);
}
```

Remarks:

- quicksort is now taking two arguments using pointer arithmetic. no need to give left and right index. only start pointer and array size is enough. we follow Lomuto partition using strcmp for comparison. swapping char *s now so the type of temp chances accordingly.
- inside readlines, we need to malloc each line[i] char * to hold enough bytes before writing into it using strcpy. runtime error occurs without proper allocation. size is len + 1 to accommodate the ending null char.
- afree must be done in reverse order of alloc. but qsort destroys initial order of alloc. so, we separately remember the initial order of allocated array in `init_lines` and afree in the opposite order.
- exit in stdlib.h allows us to terminate the program even from non-main functions. pass 0 for success ad 1 for failure.

4.1 malloc and free

in practice, you don't need to implement and use alloc and afree since there are equivalent functions malloc and free in stdlib.h. their signatures and uses are same as in our alloc and afree. however, the former two need not be called in reverse/LIFO order. The caveat is that if we don't follow this ordering, it leads to memory fragmentation and sub-optimal memory usage. with malloc and free, our readlines and main changes as follows. We no longer need the init_lines array and the copy_lineptr functions.

```
int readlines(char *lines[], int max){
    char line[LINESIZE];
    int len, i = 0;
    while(i < max && (len = get_line(line, LINESIZE)) > 0){
        lines[i] = malloc((len + 1) * sizeof(char));
        if(lines[i] == NULL){
            printf("ERROR: not enough memory to store lines\n");
            for(int j = 0; j < i; j++)
                free(lines[j]);
            exit(1);
        }
        str_cpy(lines[i], line);
        i++;
    }
    return i;
}
```

```c
int main(){
    char *lines[MAXLINES];
    int n = readlines(lines, LINESIZE);
    copy_lineptr(init_lines,lines, n);
    qsortlines(lines, n);
    printlines(lines, n);

    //free now
    for(int i = 0; i < n; i++){
        free(init_lines[i]);
        init_lines[i] = NULL;
    }
}
```

Remarks:

- it is a good practice to explicitly free explicitly all lines until i before exiting if lines[i] could not be allocated.
- free calls can happen in any order